
Paradigmas de Programación

EIF400-II-2026- CLoria

Escuela de Informática – Universidad Nacional (UNA)

Proyecto Grupal: openCypher/GQLFAuna

Versión 0.1

Trabajo Investigación Grupal: Semántica de pattern-matching en openCypher / GQL

Reporte de investigación de preparación del proyecto del curso

0. Contexto

Diseñar e implementar un lenguaje, aunque sea uno de *juguete*, obliga a usar en la práctica casi todo lo que un currículo de ciencias de la computación suele enseñar por separado: gramáticas y parsers, estructuras de datos, algoritmos de búsqueda, y los tres paradigmas de programación que se cubren en este curso.

Por otro lado, las bases de datos de grafos constituyen hoy una reconocida familia dentro del ecosistema `NoSQL`. Son usadas en producción para problemas donde el modelo relacional se vuelve *incómodo*: redes sociales, sistemas de recomendación, detección de fraude, grafos de conocimiento. Esto porque precisamente las relaciones constituyen **ciudadanos de primera clase** en esa clase de modelo, más allá de *joins* generados en cada consulta.

Casi cualquier herramienta práctica de desarrollo computacional, sea un `ORM` o un motor de reglas de negocio, termina necesitando algo parecido a un `lenguaje declarativo` para expresar el `qué` sin tener escribir el `cómo` obtenerlo. Este es un tema central en el curso. Usualmente toma la forma de un *lenguaje de dominio específico*, o `DSL` por sus siglas en inglés (`Domain Specific Language`).

Entender cómo se construye un lenguaje desde cero, aunque sea a pequeña escala, es lo que después permite reconocer ese mismo patrón para usarlo o evaluarlo con mejor criterio, más allá de usarlo simplemente como *caja negra*. El patrón de `DSL` lo encontramos en herramientas reales como `Cypher` de `Neo4J`, `SQL`, `CSS`, y motores basados en reglas.

Dentro de ese contexto y como el **proyecto** del curso, queremos desarrollar un mini-lenguaje de consulta para grafos desde cero de forma que mediante un ejercicio académico se

experimente con procesamiento de lenguajes y paradigmas de programación de una manera integral y orientada a resolver un problema.

La investigación que acá se especifica establece un **fundamento previo y complementario** al desarrollo pretendido en el área de lenguajes para consultas a bases de datos.

Proyecto `Cyphail`

- Al desarrollo del lenguaje por desarrollar lo denominamos en este documento como **el proyecto**. Tiene por nombre clave `Cyphail`.
- El proyecto cuenta con su propia especificación detallada en documento aparte: ver en [Spec Cyphail](#).

1. Objetivo de la Investigación

Antes de poder empezar a construir el lenguaje `Cyphail` de consultas sobre grafos, se requiere estudiar cómo un *lenguaje real* resuelve ese mismo problema y que se constituye en la pregunta central de la investigación. A saber:

¿Cómo hacer pattern-matching declarativo contra un grafo?

Se espera entonces una investigación **exploratoria** que les permita responder a la pregunta con suficiente profundidad: en sus propias palabras, a través de ejemplos de elaboración propia, cómo funciona la *semántica operativa* de un `MATCH` en `Cypher/GQL` y usando una **implementación real** asignada de este.

Tal implementación se tomará de una lista de implementaciones de `Cypher/GQL` la que se especifica adelante y donde cada grupo se concentra en la implementación que se le asigne de manera aleatoria.

2. Enlace con el Proyecto `Cyphail`

Este reporte es el fundamento que prepara de forma directa las fases del proyecto `Cyphail`, según la siguiente tabla:

Lo que se debe investigar	Uso en el proyecto
Semántica formal de patrones <code>(a) - [:REL] -> (b)</code>	Gramática, diseño del AST y del parser

Lo que se debe investigar	Uso en el proyecto
Cómo se resuelven patrones con múltiples relaciones encadenadas	Diseño del motor de matching / codegen
Cómo se manejan las propiedades y su tipado	Esquema de hechos Prolog
Cómo se resuelven caminos de longitud variable (<code>*</code>)	Caso 7 de su gramática

Al final del reporte se espera explícitamente que conecten lo que investigaron con las decisiones de diseño concreta de su propio proyecto, las cuales se indican en el SPEC de `Cyphail`.

3. Implementación Asignada

Cada grupo trabaja el mismo tema base (`openCypher/GQL`), pero construye sus ejemplos usando **una implementación real distinta**, asignada aleatoriamente en clase, la cual grupo estudia y sobre la que basa su reporte, que es el entregable.

Implementación	Referencia
<code>Neo4j</code>	Implementación de referencia clásica de <code>Cypher</code>
<code>Memgraph</code>	Motor en memoria, compatible con <code>Cypher</code>
Amazon <code>Neptune</code> (<code>openCypher</code>)	Servicio administrado, foco en escalabilidad
Apache <code>AGE</code>	Extensión de PostgreSQL: Cypher sobre un motor relacional
<code>Kuzu</code>	Motor embebido, orientado a análisis, open source
<code>FalkorDB</code>	Motor optimizado para IA y GraphRAG
<code>PuppyGraph</code>	Motor que virtualiza grafos sobre datos relacionales

(El profesor asignará la implementación por equipo antes de iniciar el primer sprint)

4. Preguntas que el Reporte debe Contestar

1. ¿Qué problema resuelve `Cypher/GQL` que no resolvía *bien* el `SQL` para datos con forma de grafo?
2. ¿Cómo se define formalmente la semántica de un patrón `MATCH` ? Es decir, ¿qué significa exactamente que un patrón "*matchea*" contra el grafo?
3. En la implementación asignada al grupo: ¿cómo se resuelve un patrón con **más de una relación encadenada** (equivalente al caso 5 del proyecto, el "*join implícito*")?
4. ¿Cómo maneja la implementación asignada los **caminos de longitud variable** (`*1..3`, `*`)? ¿Qué garantías hay sobre terminación y grafos con ciclos?
5. ¿Qué decisión de diseño de la implementación asignada les parece **contraria** a los alcances proyecto, y por qué?

5. Estructura Sugerida del Reporte

Refiérase al SPEC del proyecto `Cyphail`.

1. **Introducción** (3 páginas): qué es Cypher/GQL, por qué existen. Ejemplos.
2. **Semántica de pattern-matching** (2 páginas): Respuestas a las preguntas 1 y 2.
3. **Ejemplo trabajado a mano** (3 páginas) : ver sección 6 adelante.
4. **Caso de estudio: la implementación asignada** (2 páginas): respuestas a las preguntas 3 y 4.
5. **Conexión con el proyecto** (1 página): Respuesta a la pregunta 5.
6. **Referencias Usadas**: Ordenadas alfabéticamente formato `APA`.
7. **Anexos**: Si aplica

6. Énfasis: ejemplos propios elaborado *a mano*

Esta es la parte que más pesa en la evaluación del reporte. Intencionalmente hace más difícil que el resolverla por el *método* `copy-and-paste` usando una `IA` o tomarla sin elaboración de un blog, sin entenderla.

Requisitos mínimos del ejemplo:

- Debe ser un **grafo propio del equipo** (mínimo 10 nodos, mínimo 3 tipos de relaciones). No puede ser el grafo de un ejemplo de la documentación oficial ni de un tutorial.

- Deben mostrar **una consulta con al menos dos patrones encadenados**, y explicar paso a paso en prosa propia, clara y precisa, cómo el motor llega a cada resultado. No basta con mostrar el código y la salida.
- Deben mostrar qué pasa con esa misma consulta si el grafo tuviera un ciclo, o si un nodo no tuviera la propiedad que la consulta espera.
- En una sesión de defensa de esta actividad cada equipo va a explicar su ejemplo y demostrar el aprendizaje obtenido.

7. Aspectos Formales

- El entregable es un reporte técnico formal, impreso, formato pdf.
- Requerimientos

Modalidad	Grupal (solo equipos oficiales)
Fecha de asignación de implementación	Semana 1 (sorteo en clase)
Fecha de entrega del reporte	Semana 8
Sesión de defensa breve	Semana 8, en clase
Formato de entrega	PDF <i>impreso</i>
Extensión máxima	max. 12 páginas (sin contar portada, índices ni referencias)
Portada Formal	Universidad (logo), Escuela, Curso, Código de Grupo, Título/Sub-título implementación asignada Integrantes, Fecha
Estructura de Contenido General	Abstract, Tabla de Contenidos, Anexos. Ver Estructura Sugerida en este documento
Referencias	Deben poderse rastrear las citas en el texto (no poner citas no usadas ni falsas)
Diseño	Páginas numeradas. Secciones, subsecciones. Font texto: Arial 11, Font monoespaciado para código o usando imágenes

	legibles (<i>captioned</i>). Espaciado sencillo (1.5). Margen 2.5. Cualquier imagen debe indicar fuente.

8.Criterio de Evaluación

Criterio	Valor %	Criterios
Comprensión de la semántica de pattern-matching	25%	Explicación correcta y en palabras propias, no transcripción de la documentación
Ejemplos trabajados a mano	30%	Grafo propio, patrones encadenados, explicación paso a paso, manejo de casos límite
Caso de estudio de la implementación asignada	20%	Respuestas específicas a la implementación, no genéricas sobre "Cypher en general"
Conexión explícita con el proyecto programado	15%	Decisión de diseño concreta identificada y justificada
Formato, claridad y fuentes	10%	Extensión respetada, fuentes rastreables, prosa clara

Notas:

- La defensa puede ajustar la nota hacia abajo si el equipo no puede explicar satisfactoriamente su propio ejemplo sin leerlo.
- Incumplimiento en la forma y tiempo de entrega anula el trabajo.